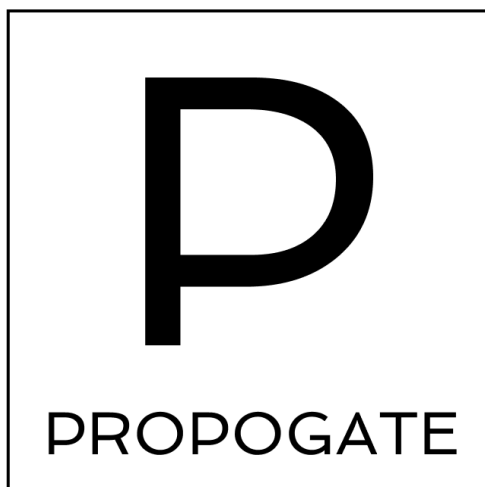




Propagate

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v0.1

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Mateo Hernán	Cornejo	62.722	macornejo@itba.edu.ar
Ignacio Gastón	Frund	61.465	ifrund@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [Flex-Bison-Compiler](#).

3. Dominio

Desarrollar un lenguaje que genere diagramas de compuertas lógicas a partir de fórmulas de lógica proposicional. El mismo debe admitir todo tipo de fórmulas válidas de dicha lógica y transformar esa entrada en un circuito lógico equivalente.

Las fórmulas se ingresarán en formato LaTeX con la posibilidad de asignar valor a las variables correspondientes, luego se analizarán, se reemplazarán los conectivos por las compuertas equivalentes y finalmente se generará un archivo de salida por cada fórmula, de manera que el usuario pueda elegir cuáles de ellas visualizar.

Esta salida también será un archivo LaTeX, para generar los diagramas utilizando el package `circuitikz`.

La implementación de este lenguaje facilitará el aprendizaje de la lógica proposicional al permitir una visualización cómoda de la valuación de las fórmulas, además de ofrecer una sintaxis más sencilla que aquella utilizada por el package para generar estos circuitos.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Declaración de variables proposicionales, con símbolos del alfabeto latino, a las que sólo se les puede asignar un valor numérico entre 0 y 1, su valuación.
- (II). Declaración de fórmulas proposicionales, con símbolos del alfabeto griego, a las que sólo se les puede asignar otras fórmulas.
- (III). Uso de conectivos lógicos unarios: `whitespace` (sin cambios) y `¬`.
- (IV). Uso de conectivos lógicos binarios: `∧`, `∨`, `→`.
- (V). Uso de paréntesis para correctitud del uso de conectivos binarios: `( )`.
- (VI). Creación de archivos de salida `.tex`, uno para cada fórmula ingresada.
- (VII). Uso de archivos de salida para generación de fórmulas compuestas.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Input de una sola variable.
- (II). Input de múltiples variables sin una fórmula.
- (III). Input de una fórmula con un conectivo unario y una variable sin paréntesis.
- (IV). Input de una fórmula con un conectivo unario y una variable con paréntesis.
- (V). Input de una fórmula con un conectivo binario y dos variables con paréntesis al inicio y final.
- (VI). Input de una fórmula con dos conectivos binarios y tres variables (anidada).

- (VII). Input de múltiples fórmulas y sus respectivas variables.
- (VIII). Input de tres fórmulas donde una de ellas es el resultado de un conector binario entre las otras dos, y sus respectivas variables.
- (IX). Input de una fórmula con dos conectivos unarios y una variable, con paréntesis correspondientes.
- (X). Input sin variables ni fórmulas.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un input con una variable que tiene asignada una fórmula.
- (II). Un input con una variable que tiene asignado un valor numérico que no sea 0 o 1.
- (III). Un input con una fórmula que tiene asignada un valor numérico.
- (IV). Un input con un conector binario donde falta una de las fórmulas.
- (V). Un input con un conector binario sin paréntesis.
- (VI). Un input con dos fórmulas consecutivas sin conector intermedio.
- (VII). Un input con dos conectivos binarios consecutivos.
- (VIII). Un input con un conector unario sin fórmula a su derecha.
- (IX). Un input con una fórmula que usa una variable no declarada.

6. Ejemplos

1. Fórmula Simple

Salida esperada: diagrama de una compuerta **AND** con entradas **p1**, **p2** y salida **\delta**.

```
// Hola
p1 = 0;
p2 = 1;
\delta = (p1 \wedge p2);
\delta.out;
```

2. Fórmula Compuesta

Salida esperada: circuito con una compuerta **OR** que combina una **IMPLIES** (**p1** $\rightarrow$ **p2**) y una **AND** (**p1** $\wedge$ **p3**):

```
// Hola
p1 = 1;
p2 = 0;
```

```
p3 = 1;  
\epsilon = ( (p1 \rightarrow p2) \vee (p1 \wedge p3) );  
\epsilon.out;
```